



Asignatura:

FULLSTACK I

Integrantes:

Camilo Poblete
Nicolás Carrasco
Carlos López

Docente:

Eduardo Baeza Lagos

Fecha:

26/05/25

1. Índice Contenidos

1. Índice Contenidos	2
2. Introducción	3
3. Diagrama de arquitectura de microservicios	4
3.1 Sobre el diagrama	4
3.2 Diagrama de despliegue	4
4. Estructura del proyecto	5
4.1 Packages	5
4.1.1 Package Controller	5
4.1.2 Package Entities	7
4.1.3 Package Repository	8
4.1.4 Package Services	9
4.2 Otros componentes importantes	10
4.2.1 Pom	10
5. Base de datos	10
5.1 Xampp	10
5.1.1 Iniciar servidor local	10
5.2 MySQL Workbench y administración de base de datos	11
5.2.1 Iniciar conexión con MySQL	11
5.2.2 Configuración nueva conexión	12
5.2.3 Ingreso a nueva conexión	13
5.2.4 Crear nueva base de datos	13
5.2.5 Spring Boot dashboard	14
6. Implementación de los servicios	15
6.1 Ruta de los servicios	16
6.2 Solicitud Post	16
6.3 Solicitud Get	17
6.4 Solicitud Put	18
6.5 Solicitud Delete	19
7. Maven	20
8. Git Hub	21
8.1 Git Init	21
8.2 Git Config	21
8.3 Git Add y Git Commit	24
8.4 Git Remote, Git Branch, Git Checkout y Git Push	25
8.5 Git Merge	26
9. Conclusión	28

2. Introducción

Este informe es una continuación de nuestro proyecto de microservicios para la empresa EduTech Innovators. Empezaremos mostrando el diagrama de despliegue, el cual nos señala la arquitectura de microservicios que implementaremos.

Luego pasaremos a la estructura del proyecto, donde a través de imágenes, mostraremos los distintos componentes que lo estructuran a nivel de código (utilizaremos Visual Studio Code como IDE).

Más adelante ahondaremos en el apartado de la base de datos, donde crearemos las tablas y las poblaremos con los datos para luego poder manipularlos mediante los microservicios. Utilizaremos como motor de base de datos a MySQL por su versatilidad y robustez.

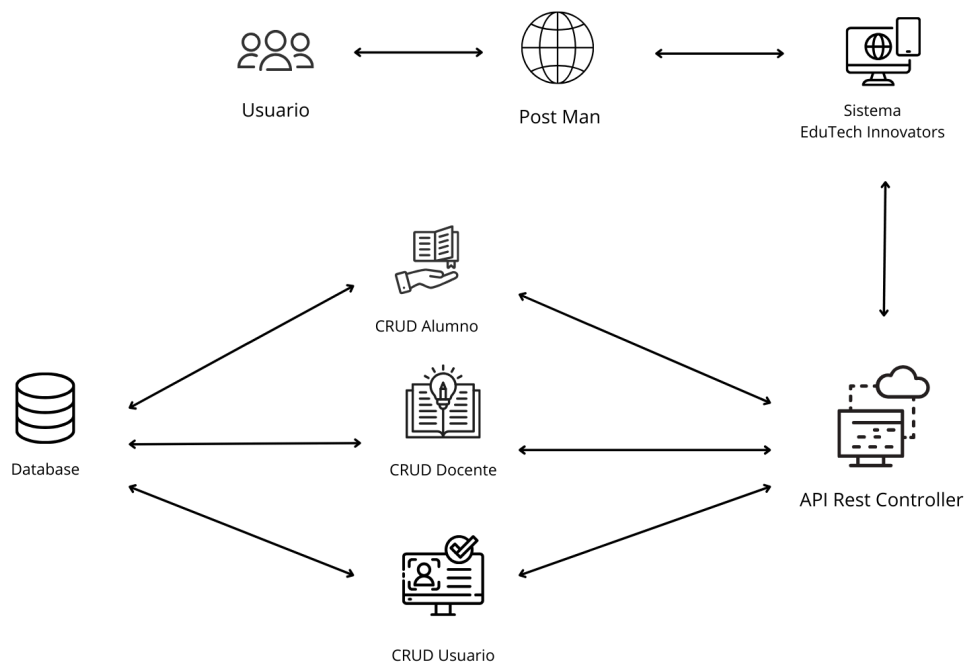
Luego pasaremos a la implementación de los servicios del sistema y las vistas para que se puedan visualizar en pantalla y finalmente, daremos término a nuestro informe recopilando los documentos en la plataforma github a través de comandos git..

3. Diagrama de arquitectura de microservicios

3.1 Sobre el diagrama

En nuestro informe anterior (primera unidad) desarrollamos tres diagramas distintos. El diagrama de clases, el diagrama de casos de uso y el diagrama de despliegue. Este último diagrama resulta importante en este momento ya que es el diagrama que nos muestra la ruta que siguen los distintos servicios, es decir, su arquitectura. A continuación mostramos en qué consiste el diagrama de despliegue.

3.2 Diagrama de despliegue

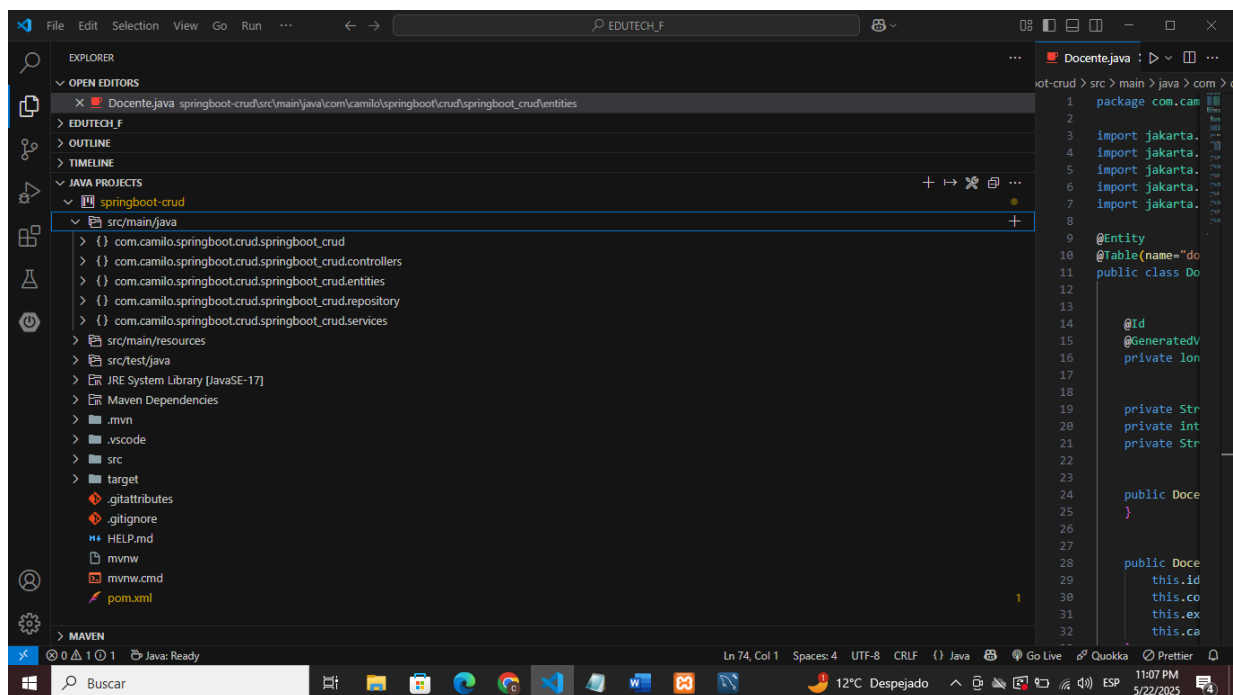


4. Estructura del proyecto

La estructura es muy importante para el proyecto. Esta estructura nos da orden y garantiza el correcto funcionamiento de todas las partes del software. Usaremos el IDE Visual Studio Code para codificar los distintos elementos.

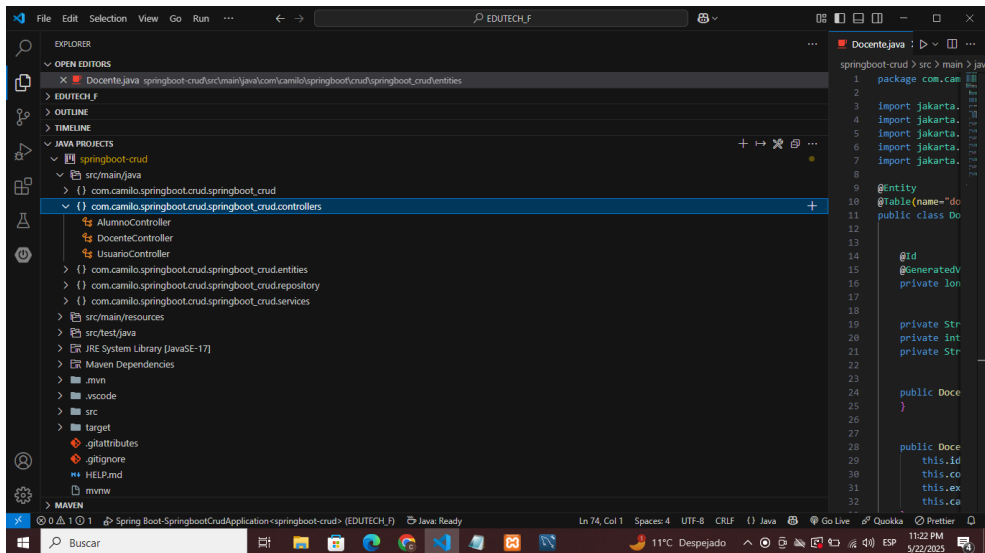
4.1 Packages

Nuestro proyecto consta básicamente de cinco packages. El package principal del proyecto *springboot_crud*, el package *controllers*, el package *entities*, el package *repository* y el package *services*. Estos packages se ubican en la carpeta *src/main/java* del proyecto. Estos packages contienen todas las clases e interfaces para la operatividad de nuestro sistema.



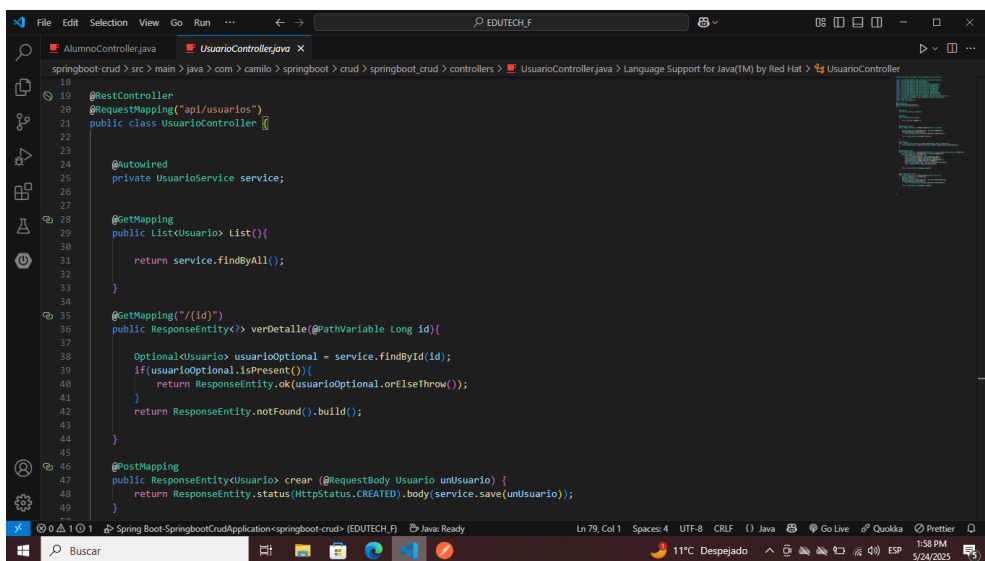
4.1.1 Package Controller

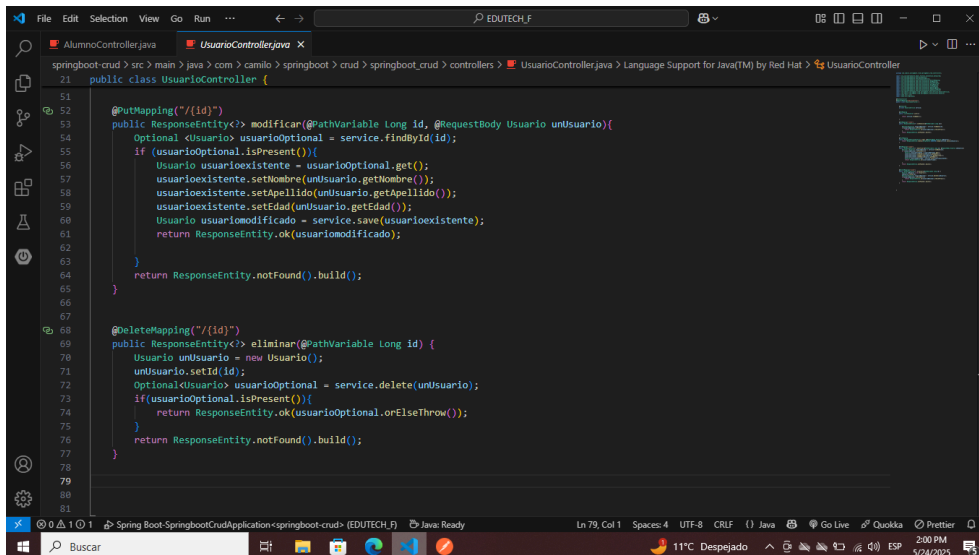
En el package controller hay tres clases. *UsuarioController*, *AlumnoController* y *DocenteController*. Estas clases son las “Controladoras”, es decir dirigen los microservicios a sus destinos para que puedan ejecutarse las órdenes del usuario.



Aquí vemos la clase UsuarioController donde se encuentra al API Rest Controller que actúa como intermediario entre las solicitudes de usuario y el sistema.

Además, más abajo se encuentran todos los servicios que el sistema como GET, POST, PUT Y DELETE.

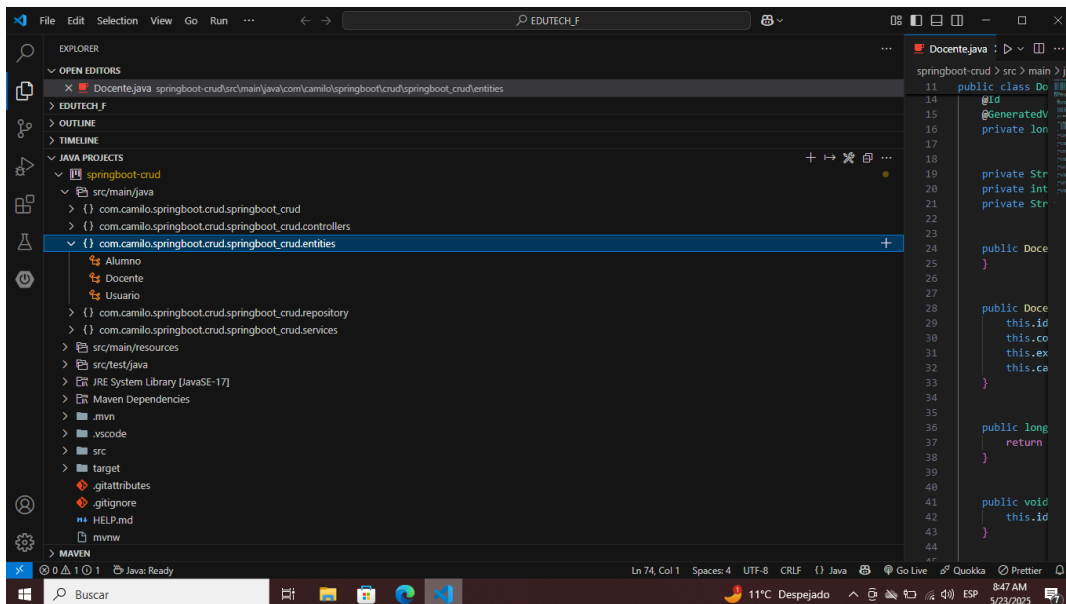




```
21 public class UsuarioController {
22
23     @PutMapping("/{id}")
24     public ResponseEntity<> modificar(@PathVariable Long id, @RequestBody Usuario unUsuario){
25         Optional<Usuario> usuarioOptional = service.findById(id);
26         if (usuarioOptional.isPresent()){
27             Usuario usuarioExistente = usuarioOptional.get();
28             usuarioExistente.setNombre(unUsuario.getNombre());
29             usuarioExistente.setApellido(unUsuario.getApellido());
30             usuarioExistente.setEdad(unUsuario.getEdad());
31             Usuario usuarioModificado = service.save(usuarioExistente);
32             return ResponseEntity.ok(usuarioModificado);
33         }
34         return ResponseEntity.notFound().build();
35     }
36
37     @DeleteMapping("/{id}")
38     public ResponseEntity<> eliminar(@PathVariable Long id) {
39         Usuario unUsuario = new Usuario();
40         unUsuario.setId(id);
41         Optional<Usuario> usuarioOptional = service.delete(unUsuario);
42         if(usuarioOptional.isPresent()){
43             return ResponseEntity.ok(usuarioOptional.orElseThrow());
44         }
45         return ResponseEntity.notFound().build();
46     }
47 }
```

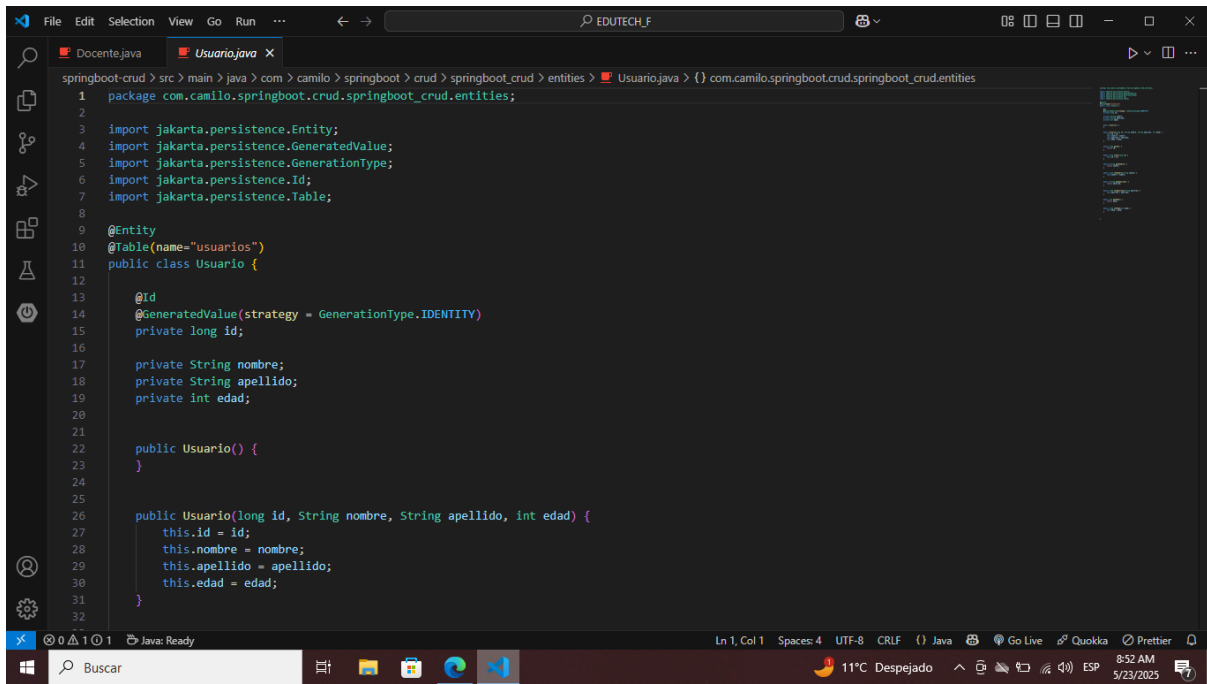
4.1.2 Package Entities

En el package entities hay tres clases. *Usuario*, *Alumno* y *Docente*. Estas clases representan las entidades principales de nuestro proyecto en Edutech. Todas poseen atributos que les son propios. Además de constructores y sus respectivos getters y setters.



```
11 public class Docente {
12     @Id
13     @GeneratedValue
14     private Long id;
15
16     private String nombre;
17     private Integer edad;
18     private String apellido;
19
20     public Docente() {
21     }
22
23     public Docente(
24         this.id,
25         this.nombre,
26         this.edad,
27         this.apellido
28     ) {
29     }
30
31     public Long getId() {
32         return id;
33     }
34
35     public void setId(Long id) {
36         this.id = id;
37     }
38 }
```

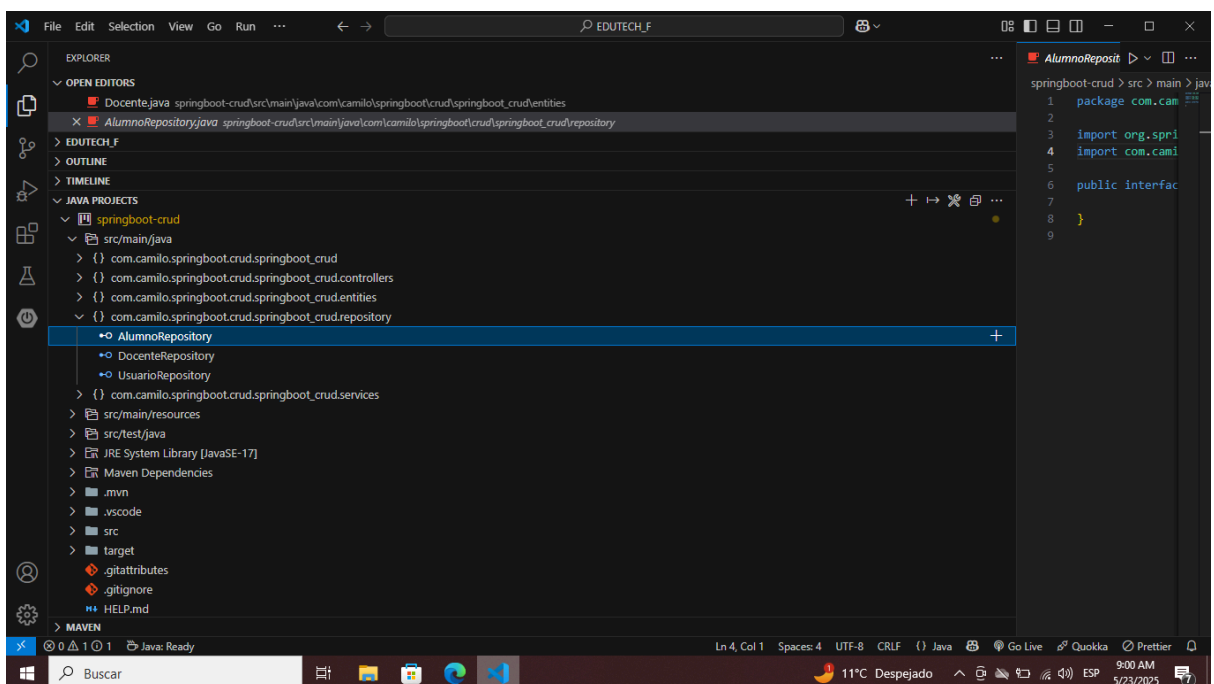
A modo de ejemplo tenemos la clase o entidad usuario. Esta posee los atributos id, nombre, apellido y edad. Además en esta clase se define el nombre que tendrá la tabla asociada en la base de datos a través de la marca `@Table`.



```
1 package com.camilo.springboot.crud.springboot_crud.entities;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.GeneratedValue;
5 import jakarta.persistence.GenerationType;
6 import jakarta.persistence.Id;
7 import jakarta.persistence.Table;
8
9 @Entity
10 @Table(name="usuarios")
11 public class Usuario {
12
13     @Id
14     @GeneratedValue(strategy = GenerationType.IDENTITY)
15     private long id;
16
17     private String nombre;
18     private String apellido;
19     private int edad;
20
21     public Usuario() {
22     }
23
24     public Usuario(long id, String nombre, String apellido, int edad) {
25         this.id = id;
26         this.nombre = nombre;
27         this.apellido = apellido;
28         this.edad = edad;
29     }
30
31 }
```

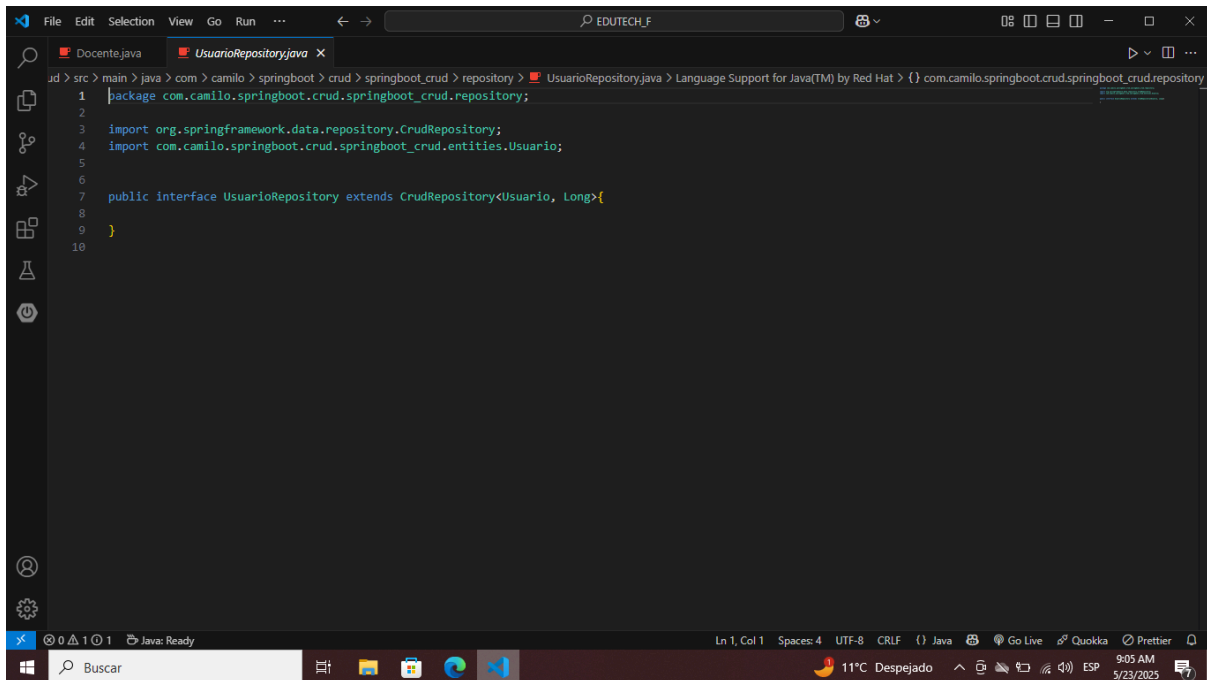
4.1.3 Package Repository

En el package *Repository* se almacenan las interfaces *UsuarioRepository*, *AlumnoRepository* y *DocenteRepository*. Estas interfaces permitirán más adelante manejar la lógica de acceso a la base de datos generando automáticamente el código SQL.



```
1 package com.camilo.springboot.crud.springboot_crud.repository;
2
3 import org.springframework.data.jpa.repository.JpaRepository;
4
5 public interface AlumnoRepository extends JpaRepository<Alumno, Long> {
6
7 }
8
9 }
```

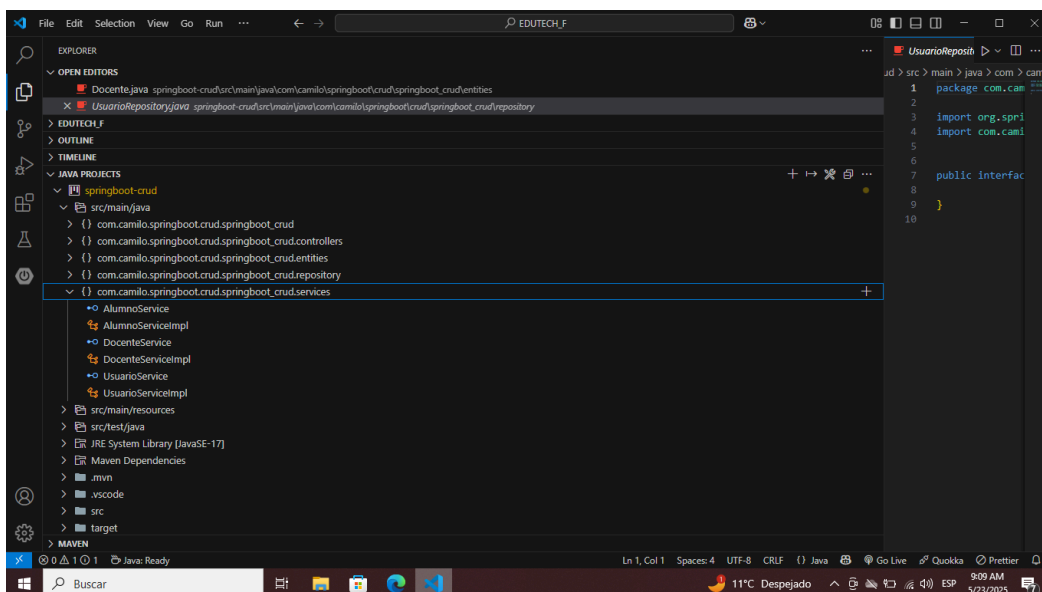

A modo de ejemplo tenemos la interface *UsuarioRepository*.



```
1 package com.camilo.springboot.crud.springboot_crud.repository;
2
3 import org.springframework.data.repository.CrudRepository;
4 import com.camilo.springboot.crud.springboot_crud.entities.Usuario;
5
6
7 public interface UsuarioRepository extends CrudRepository<Usuario, Long>{
8
9 }
10
```

4.1.4 Package Services

Finalmente tenemos el package *Service*. Al interior de este package se encuentran las clases e interfaces que contarán con los métodos que realizan los distintos servicios del sistema. Este package posee 3 clases y tres interfaces. Una de cada una por cada entidad (Usuario, Alumno y Docente).

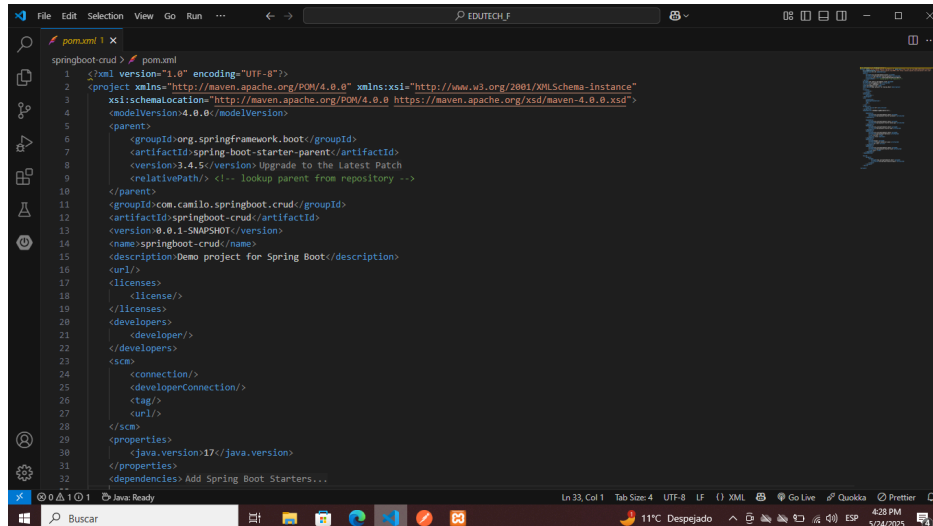


```
1 package com.camilo.springboot.crud.springboot_crud.services;
2
3 import org.springframework.data.repository.CrudRepository;
4 import com.camilo.springboot.crud.springboot_crud.entities.Usuario;
5
6
7 public interface UsuarioService {
8
9 }
10
```

4.2 Otros componentes importantes

4.2.1 Pom

El pom es fundamental. Es el archivo de configuración que usa Maven y que permite gestionar las dependencias, los plugins y la configuración general del proyecto.



5. Base de datos

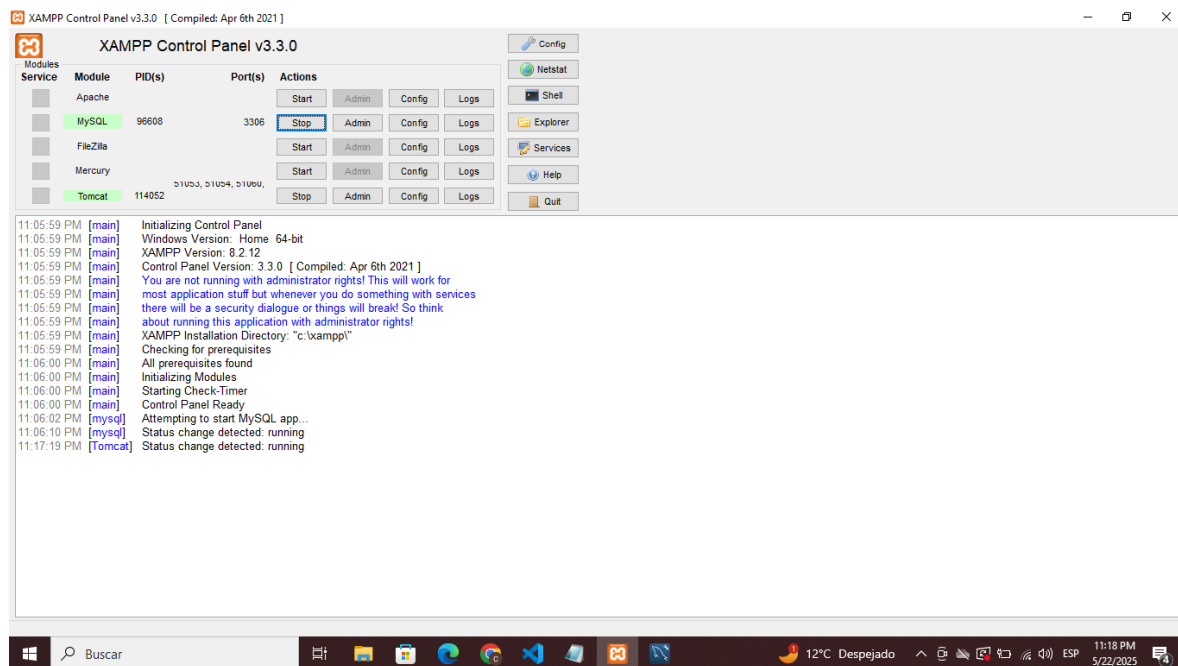
La base de datos es la que nos permitirá almacenar la información de las entidades del sistema. En nuestro proyecto utilizaremos Xampp para crear nuestro servidor local y Mysql Workbench para administrar la base de datos virtualmente.

5.1 Xampp

Xampp es un software gratuito que nos permitirá desarrollar aplicaciones web sin necesidad de un servidor en internet. nos permite levantar un servidor local en nuestra computadora. De esta manera utilizaremos Xampp para levantar nuestro propio servidor local.

5.1.1 Iniciar servidor local

Al presionar el botón start de MySQL en el panel de control de xampp levantamos nuestro servidor local MySQL en el puerto 3306.



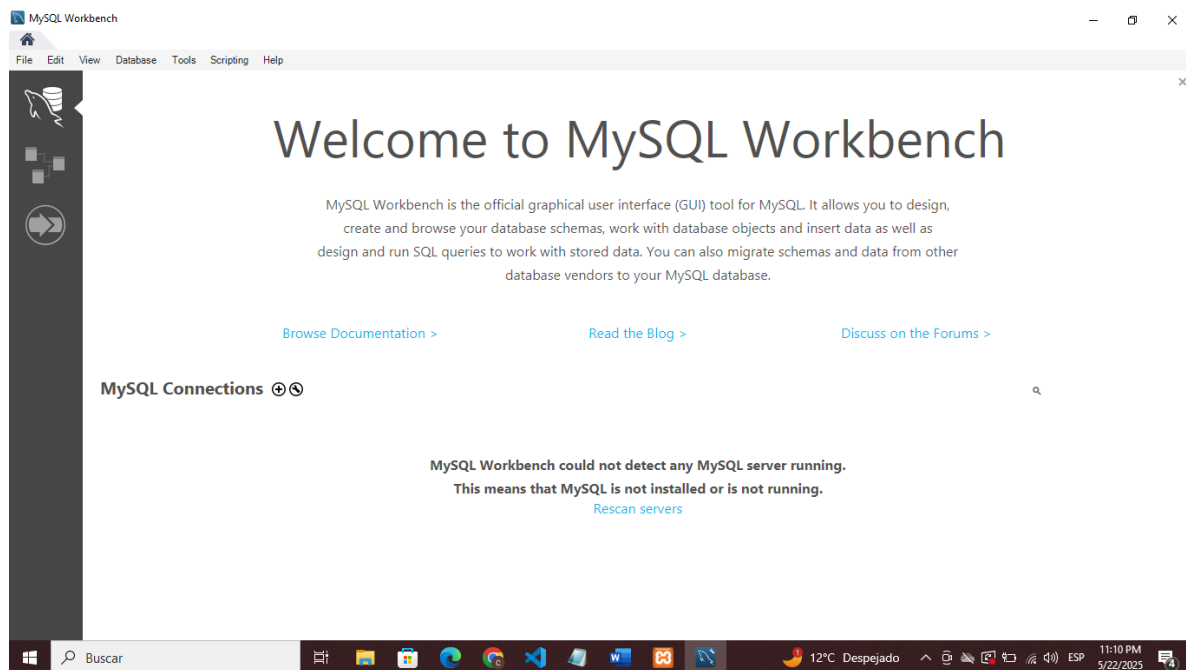
5.2 MySQL Workbench y administración de base de datos

MySQL Workbench es una herramienta de administración y diseño de bases de datos que permite conectarse a un servidor MySQL como el que arranca Xampp.

Xampp arranca el servidor de manera local y MySQL Workbench se conecta a ese servidor para administrar visualmente la base de datos.

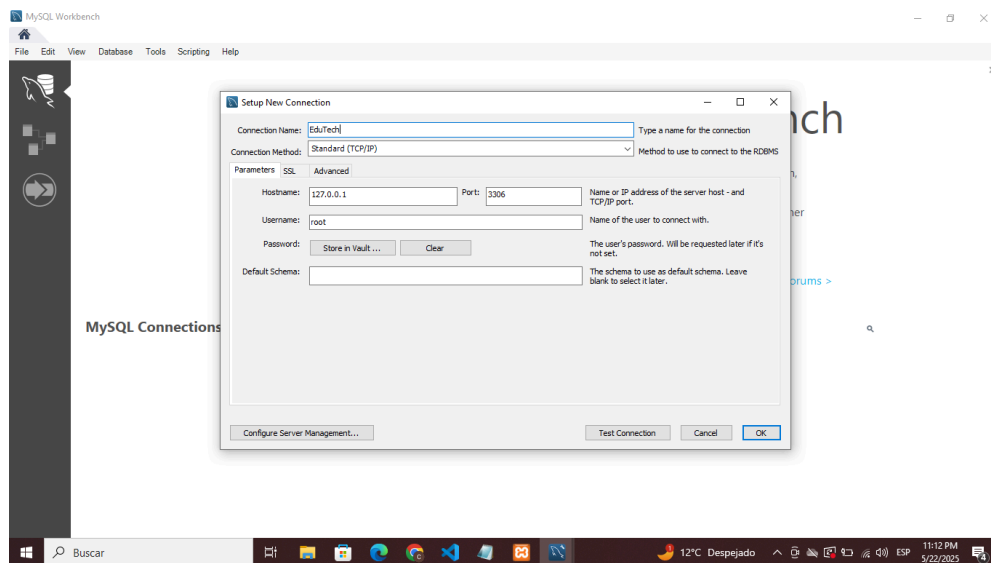
5.2.1 Iniciar conexión con MySQL

En el panel de control principal de MySQL workbench iniciamos una nueva conexión



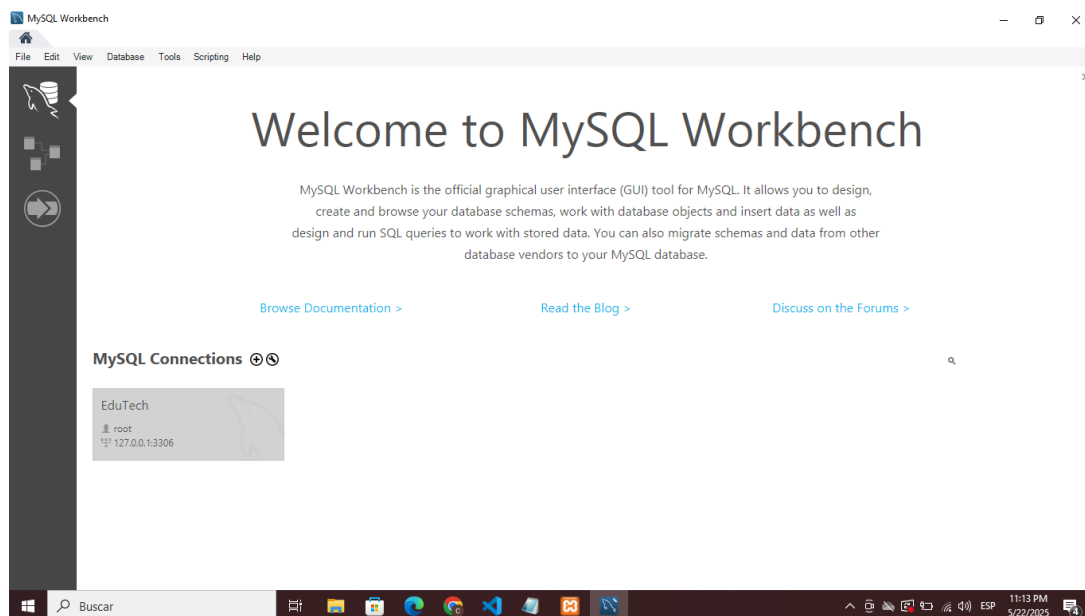
5.2.2 Configuración nueva conexión

Aquí le damos un nombre a nuestra conexión, la testamos y aceptamos.



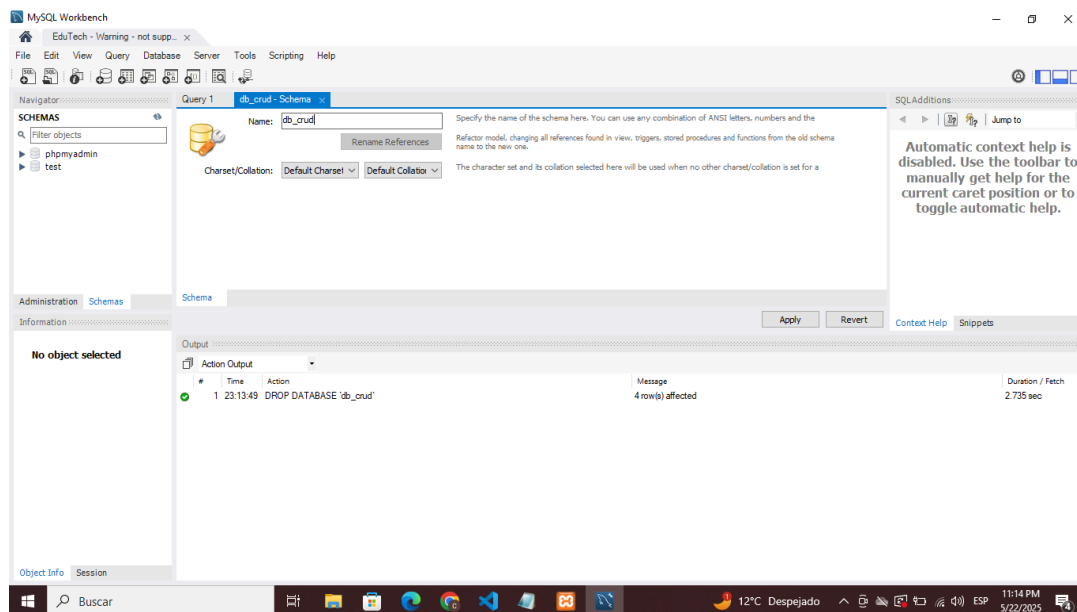
5.2.3 Ingreso a nueva conexión

Una vez creada la conexión ingresamos a ella.

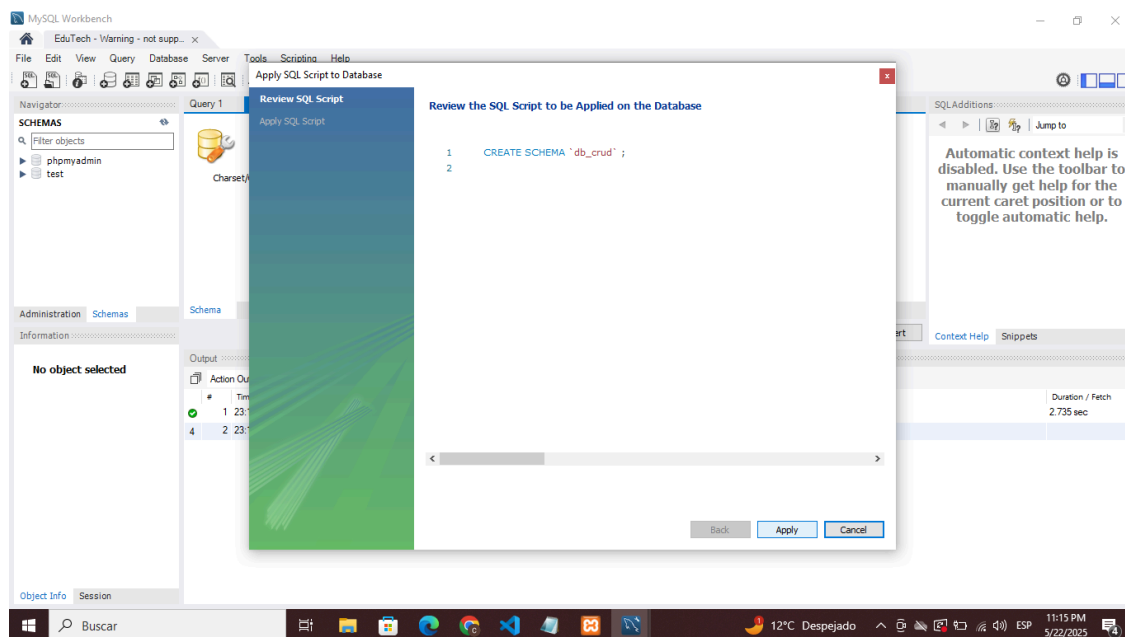


5.2.4 Crear nueva base de datos

Una vez dentro de nuestra conexión creamos una nueva base de datos y le damos un nombre.



Presionamos aplicar y aceptamos.

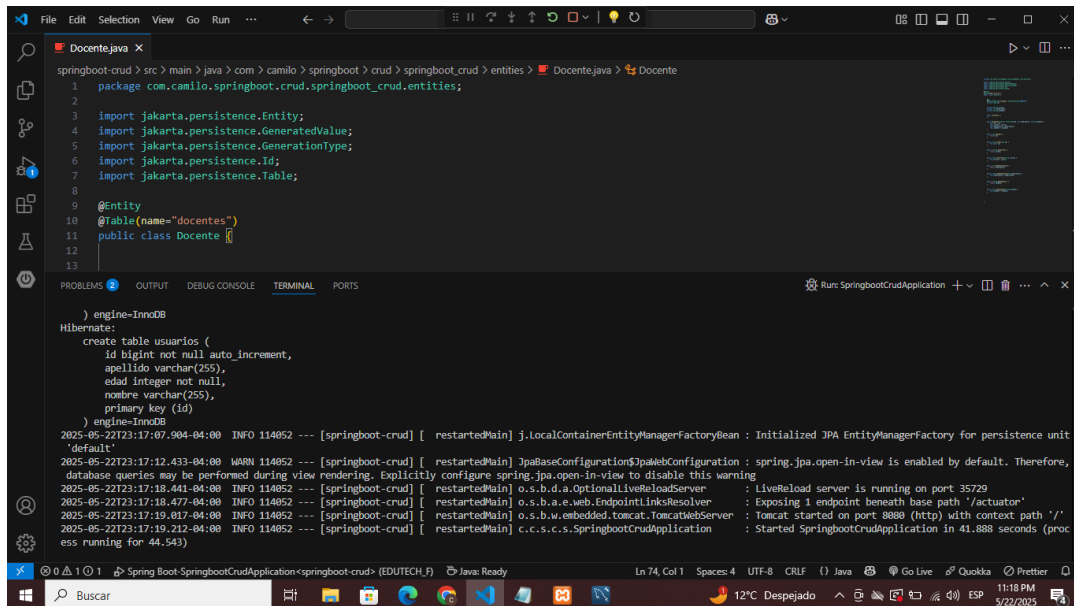


Una vez realizados estos pasos ya tenemos creada visualmente nuestra base de datos.

5.2.5 Spring Boot dashboard

Luego vamos a nuestro proyecto Spring Boot en Visual Studio Code y damos run a nuestro proyecto desde Spring Boot Dashboard.

Y como vemos se han creado las tablas vacías que previamente codificamos en el sistema.



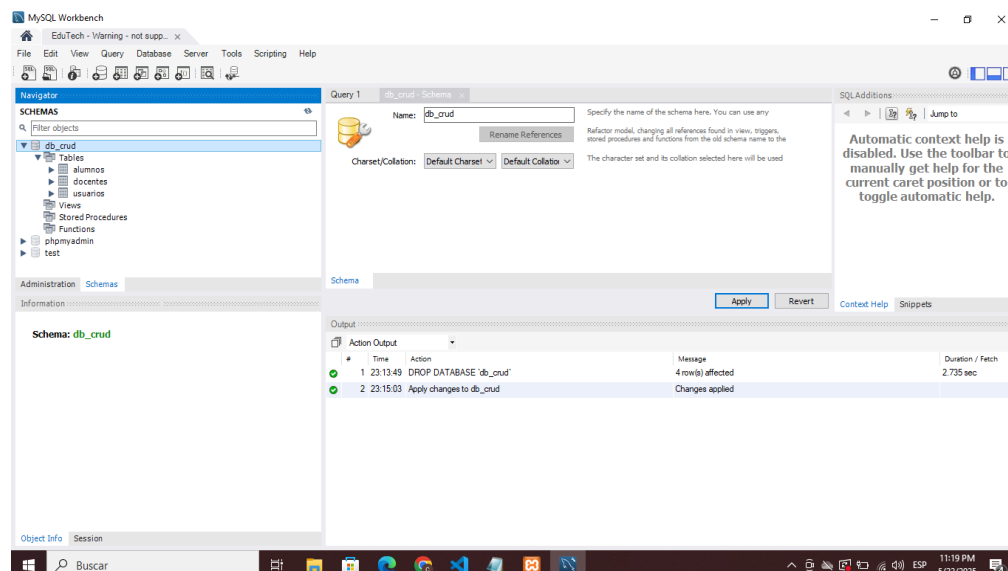
The screenshot shows an IDE with a Java file named `Docente.java` and a terminal window. The Java code defines a JPA entity `Docente` with a table named `docentes`. The terminal output shows the application starting successfully, with messages from Hibernate and Spring Boot.

```
Docente.java
1 package com.camilo.springboot.crud.springboot_crud.entities;
2
3 import jakarta.persistence.Entity;
4 import jakarta.persistence.GeneratedValue;
5 import jakarta.persistence.GenerationType;
6 import jakarta.persistence.Id;
7 import jakarta.persistence.Table;
8
9 @Entity
10 @Table(name="docentes")
11 public class Docente {
12
13
14 }

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Run: SpringBootCrudApplication

) engine-InnoDB
Hibernate:
create table usuarios (
  id bigint not null auto_increment,
  apellido varchar(255),
  edad integer not null,
  nombre varchar(255),
  primary key (id)
) engine=InnoDB
2025-05-22T23:17:07.904-04:00 INFO 114052 --- [springboot-crud] [ restarted@Main] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'
2025-05-22T23:17:12.433-04:00 WARN 114052 --- [springboot-crud] [ restarted@Main] JpaBaseConfiguration$JpaWebConfiguration : spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this warning
2025-05-22T23:17:18.441-04:00 INFO 114052 --- [springboot-crud] [ restarted@Main] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
2025-05-22T23:17:18.477-04:00 INFO 114052 --- [springboot-crud] [ restarted@Main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 1 endpoint beneath base path '/actuator'
2025-05-22T23:17:19.017-04:00 INFO 114052 --- [springboot-crud] [ restarted@Main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
2025-05-22T23:17:19.212-04:00 INFO 114052 --- [springboot-crud] [ restarted@Main] c.c.s.c.s.SpringBootCrudApplication : Started SpringBootCrudApplication in 41.888 seconds (process running for 44.543)
```

Luego vemos en MySQL Workbench que tenemos las tablas de manera visual listas para administrar.



6. Implementación de los servicios

Para la implementación de los microservicios utilizaremos la herramienta Postman. Este software nos permitirá realizar las solicitudes al sistema básicas de nuestro proyecto como

crear, leer, modificar y eliminar (CRUD) sobre las distintas entidades que tenemos (Usuario, Alumno y Docente).



6.1 Ruta de los servicios

Cuando realizamos una solicitud desde postman, enviamos un requerimiento que se comunica con la API Rest del sistema el cual redirige la solicitud según corresponda, ya sea si queremos traer, crear, modificar o eliminar un nuevo Usuario, Alumno o Docente.

Esquema:

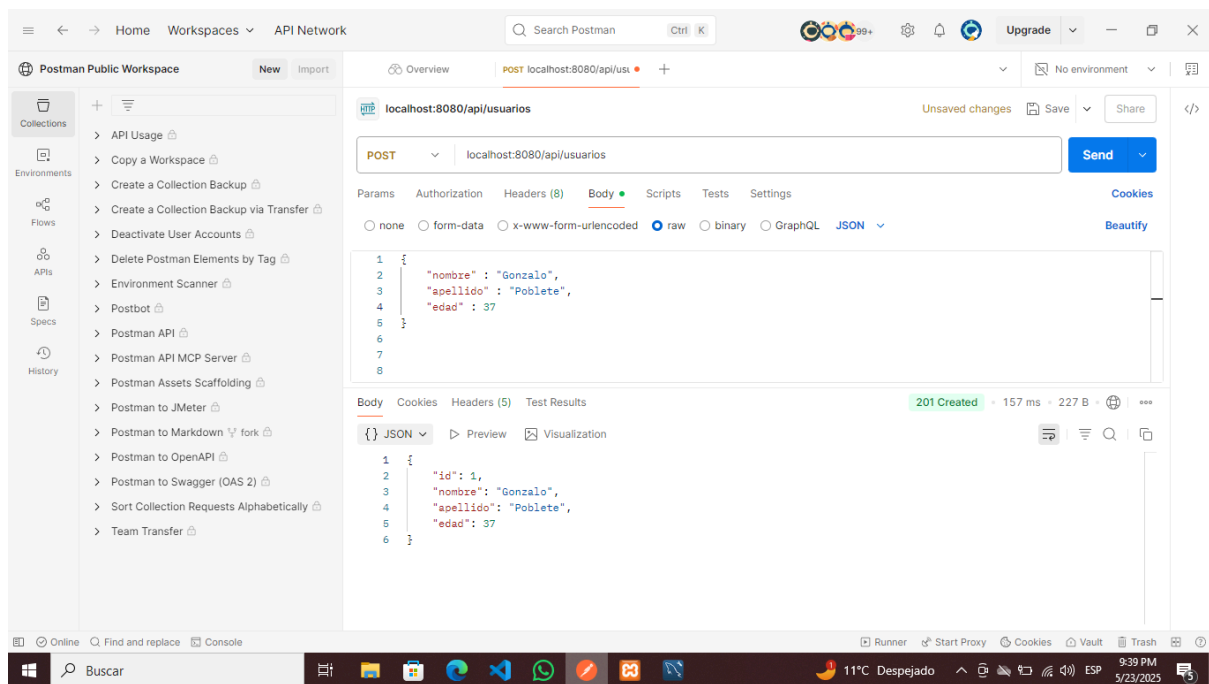
Cliente (Postman, Navegador, App móvil, etc.)



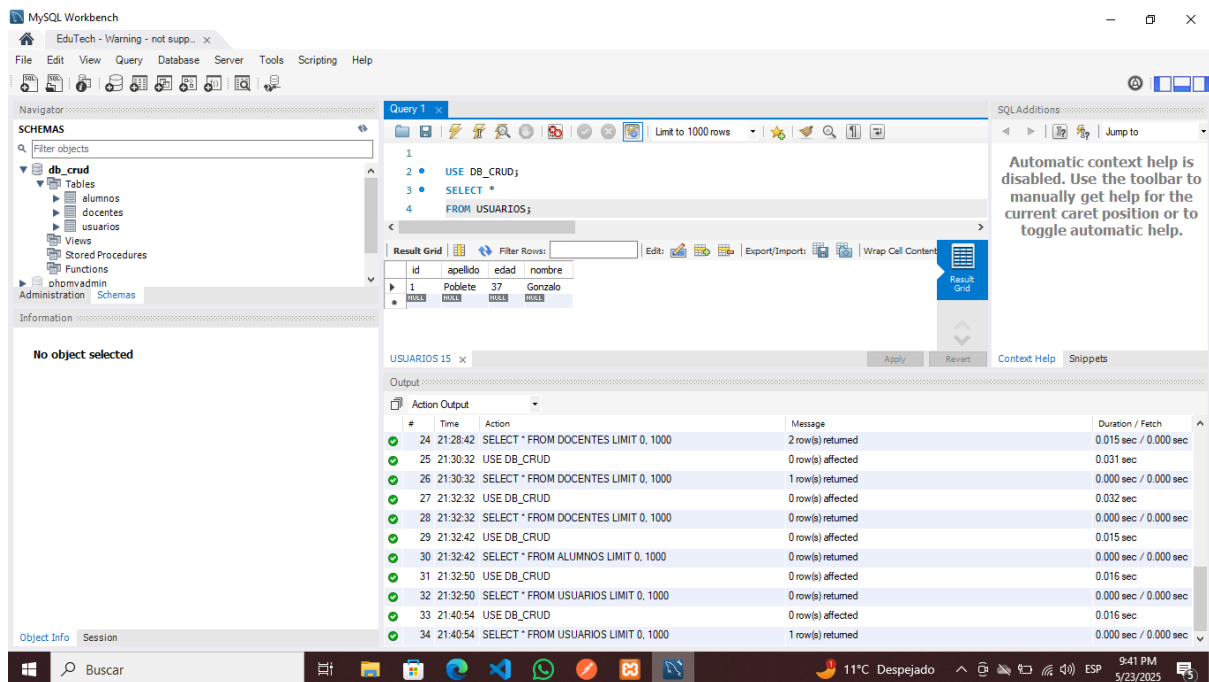
@RestController → @Service (interface) → ServiceImpl → Repository (JPA) → Base de datos

6.2 Solicitud Post

En este ejemplo hemos realizado una solicitud Post para crear un nuevo usuario.

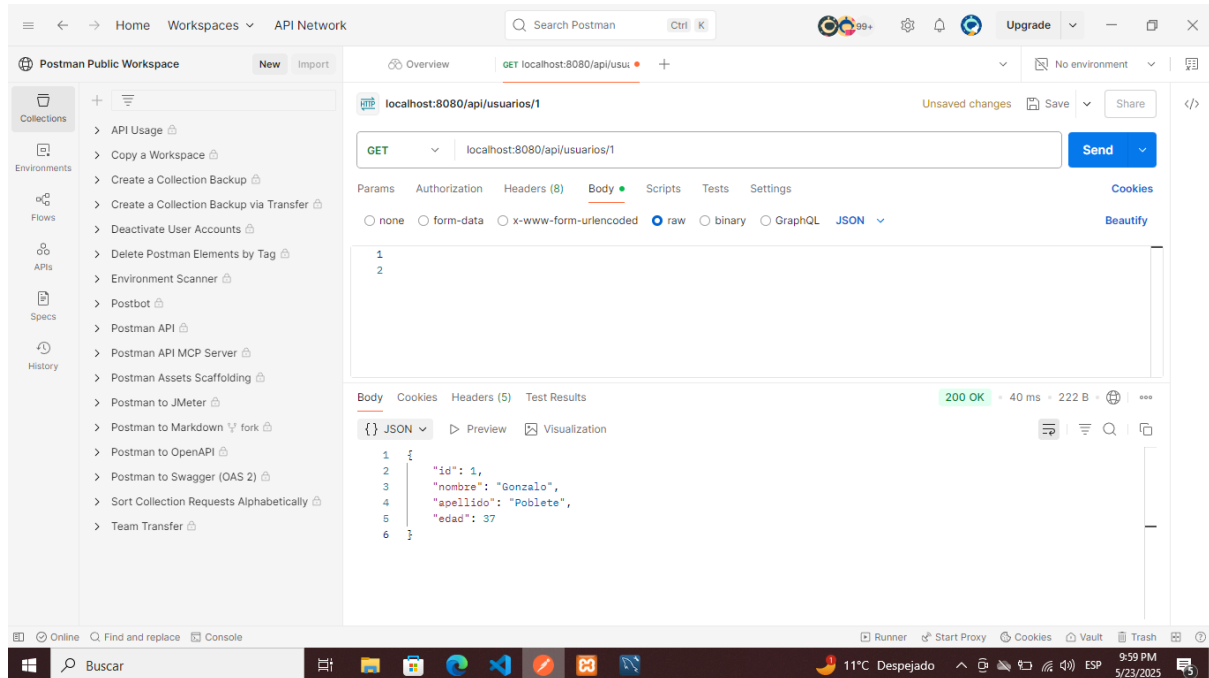


Si luego buscamos los datos en la tabla de usuarios en MySQL Workbench podemos ver que se ha almacenado la información del nuevo usuario correctamente.



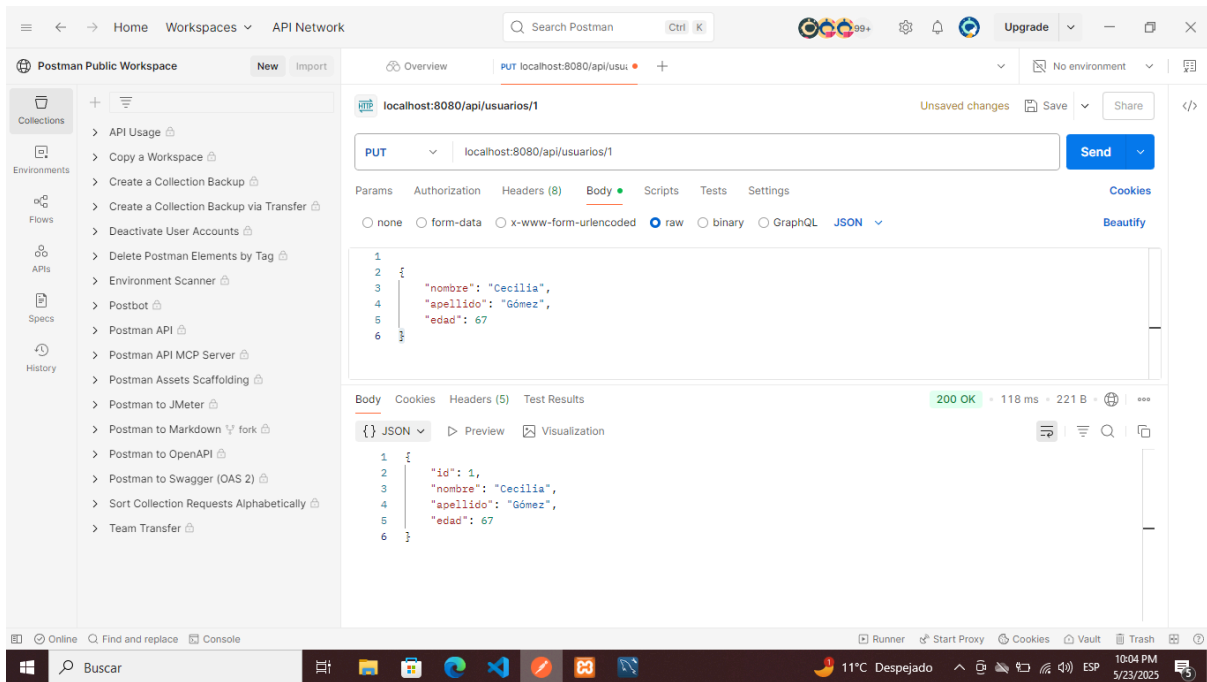
6.3 Solicitud Get

Nos permite solicitar la información de un registro en el sistema. En este ejemplo solicitamos los datos de la tabla usuarios a aquel que posee el id número 1. El sistema nos devuelve su id, nombre, apellido y edad.



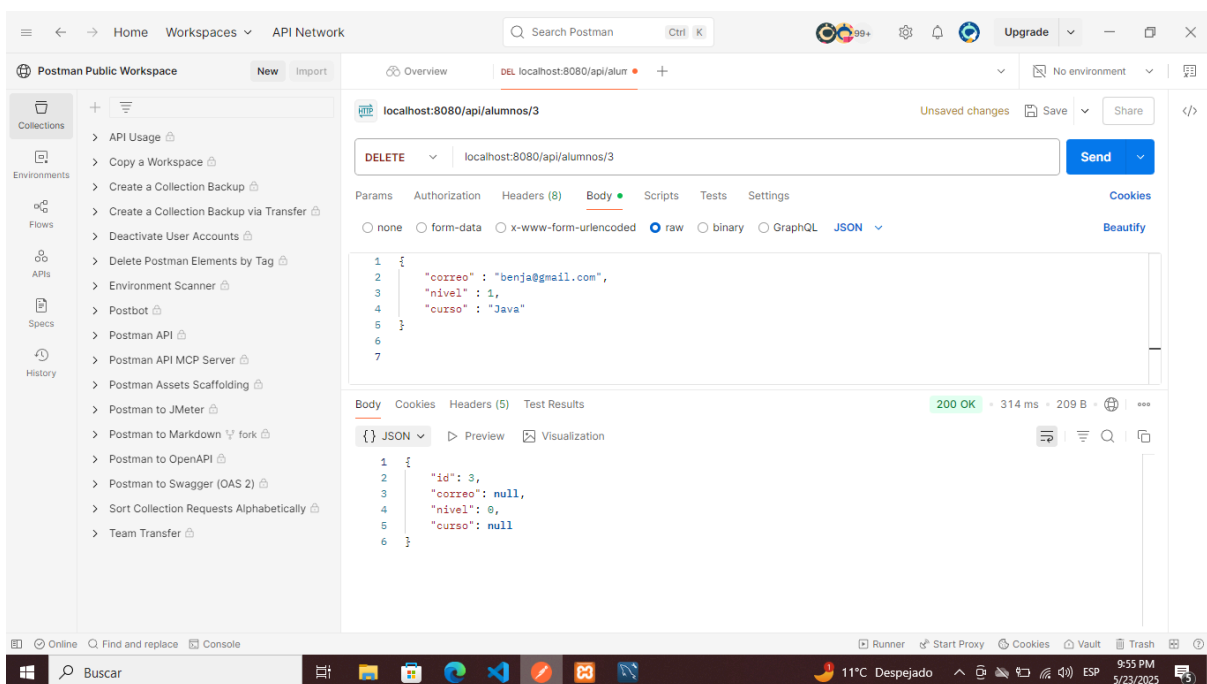
6.4 Solicitud Put

Put nos permite modificar los datos de un registro. Siguiendo con el ejemplo anterior modificaremos los datos del usuario que posee el id número 1.



6.5 Solicitud Delete

La solicitud Delete nos permite eliminar un registro del sistema. En este ejemplo vemos cómo eliminamos un registro que posee el id número 3 de la tabla alumnos.



7. Maven

Maven es una herramienta de trabajo útil para la automatización y gestión de proyectos que nos ayuda a compilar, empaquetar dependencias de forma eficiente. En nuestro caso, hizo posible la configuración primaria y la trayectoria de integración de las bibliotecas utilizadas en el proyecto ya que se cargan a través del archivo pom.xml.

Además en nuestro proyecto hecho en Spring Boot, Maven tiene un rol esencial e importante para mitigar las dependencias externas como Spring Web, Spring Data JPA y el conector de MySQL.

A continuación se muestran los aspectos más significativos del archivo pom.xml :

- La versión de java con la cual se realizó este proyecto es 17, lo que se está indicando en el mismo archivo

Principales e importantes dependencias del proyecto:

- spring-boot-starter-web: Se utilizó para crear servicios web ApiRest.
- spring-boot-starter-data-jpa: Para persistencia de datos esenciales mediante JPA.
- mysql-connector-java: conexión con la base de datos MySQL.
- spring-boot-starter-actuator: Este tiene como función exponer métricas y demás endpoints de monitoreo.
- spring-boot-devtools: Para reiniciar de forma automática durante el desarrollo
- spring-boot-starter-test: Para la creación de pruebas unitarias e integraciones.

Ejemplos:

```

34     <dependency>
35     <groupId>org.springframework.boot</groupId>
36     <artifactId>spring-boot-starter-actuator</artifactId>
37     </dependency>
38
39     <dependency>
40     <groupId>org.springframework.boot</groupId>
41     <artifactId>spring-boot-starter-data-jpa</artifactId>
42     </dependency>
43     <dependency>
44     <groupId>org.springframework.boot</groupId>
45     <artifactId>spring-boot-starter-web</artifactId>
46     </dependency>
47     <dependency>
48     <groupId>org.springframework.boot</groupId>
49     <artifactId>spring-boot-devtools</artifactId>
50     <scope>runtime</scope>
51     <optional>true</optional>
52     </dependency>
53     <dependency>
54     <groupId>mysql</groupId>
55     <artifactId>mysql-connector-java</artifactId>
56     <version>8.0.26</version>
57     </dependency>
58     <dependency>
59     <groupId>org.springframework.boot</groupId>
60     <artifactId>spring-boot-starter-test</artifactId>
61     <scope>test</scope>
62     </dependency>
63 </dependencies>

```

8. Git Hub

Nosotros utilizaremos GitHub el cuál es un servicio basado en la web. Este servicio utiliza Git, una herramienta destinada para el control de versiones de los proyectos que los programadores crean y guarda el código en la web.

Es importante aprender ciertos comandos básicos para poder trabajar en un proyecto, ya que Git nos permitirá:

- Guardar los cambios realizados a los archivos del proyecto
- Trabajar en equipo sin interferir con los cambios que cada integrante realice, utilizando distintas “ramas” para guardar sus avances.
- Volver a versiones anteriores del proyecto si algún integrante se equivocó en alguna parte del código.
- Combinar el trabajo guardado en las ramas en una sola rama.

8.1 Git Init

El comando git init nos permite inicializar un nuevo repositorio en la carpeta en la cuál se encuentra nuestro proyecto.

```
nicho@kingniki MINGW64 ~/Desktop/Ev2_EduTech
• $ git init
Initialized empty Git repository in C:/Users/nicho/Desktop/Ev2_EduTech/.git/
```

También este comando creará de forma automática una carpeta `.git` dentro de la carpeta de nuestro proyecto, la cuál contendrá toda la información para hacer la gestión de versiones e historial de archivos.

8.2 Git Config

El comando `git config` sirve para configurar las opciones y ajustes de git. Es necesario utilizar este comando, ya que tendremos que configurar el comportamiento de Git.

Hay muchas opciones que podremos configurar, pero principalmente utilizaremos solo tres comandos comunes con `git config`, los cuales necesitará Git para ver quien realiza los cambios dentro del repositorio del proyecto (commits)

`git config user.name nombre_del_usuario`: Este comando definirá el nombre del usuario al que pertenecerá a este repositorio.

```
nicho@kingniki MINGW64 ~/Desktop/Ev2_EduTech (main)
• $ git config user.name nicolas
```

`git config user.email correo_del_usuario`: Este comando definirá el correo del usuario al que pertenecerá a este repositorio.

```
nicho@kingniki MINGW64 ~/Desktop/Ev2_EduTech (main)
• $ git config user.email nico.carrascon@duocuc.cl
```

`git config user.password contraseña_del_usuario`: Este comando definirá la contraseña del usuario al que pertenecerá a este repositorio.

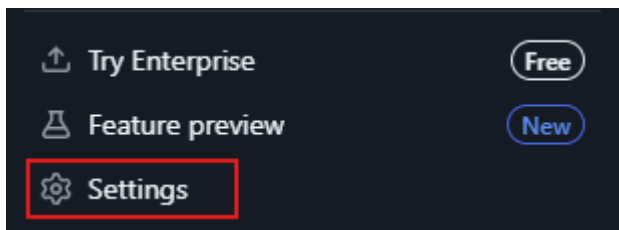
```
nicho@kingniki MINGW64 ~/Desktop/Ev2_EduTech (main)
• $ git config user.password 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
```

Para definir la contraseña nosotros decidimos generar un token en la plataforma GitHub siguiendo unos simples pasos:

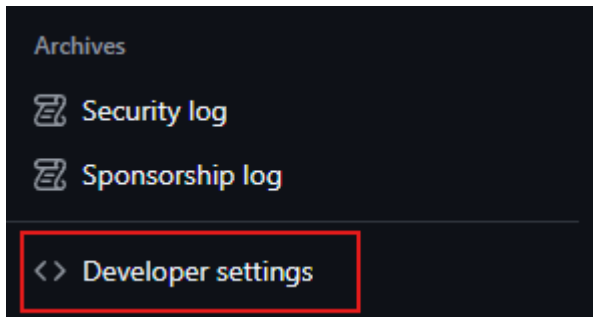
Primero debemos dirigirnos al menú del perfil de nuestro usuario en GitHub.



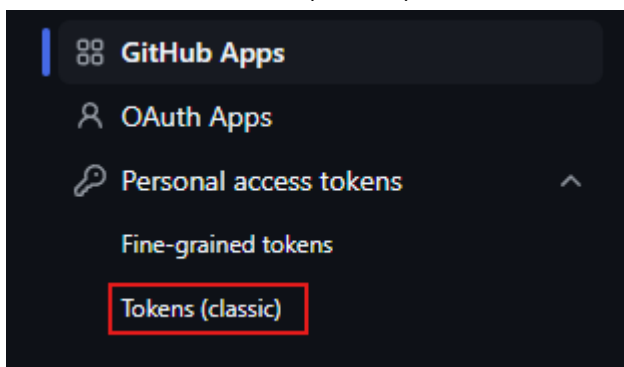
Seleccionamos la opción Settings.



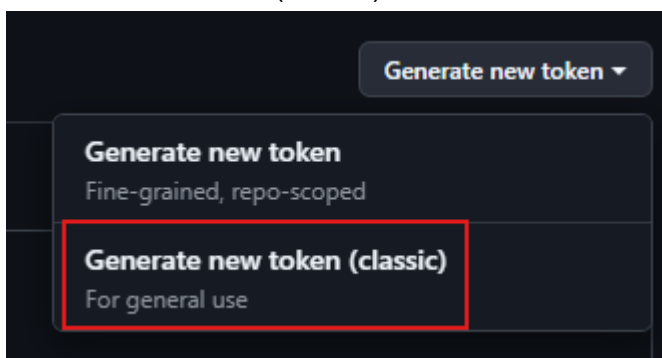
Ya en las configuraciones del perfil nosotros debemos bajar hasta la sección Archives y seleccionar la opción Developer settings



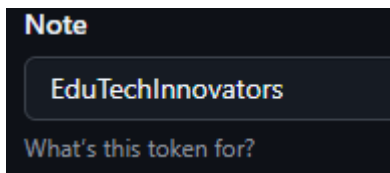
Al ingresar a las configuraciones, seleccionamos Personal access tokens y luego seleccionamos Tokens (classic)



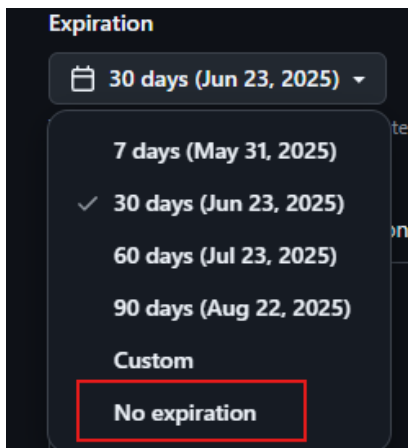
A continuación seleccionamos Generate new token para luego seleccionar la opción Generate new token (classic)



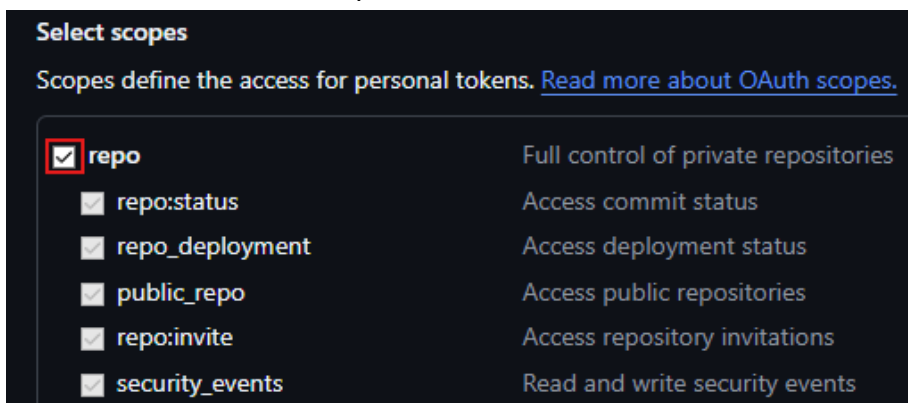
En la casilla Note ingresamos un nombre descriptivo para nuestro token



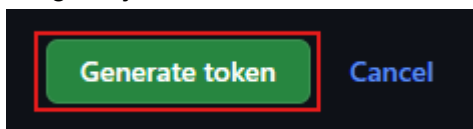
Nosotros podemos seleccionar cualquier opción de fecha de expiración de acuerdo a cuánto tiempo le dedicaremos al proyecto, en nuestro caso decidimos utilizar la opción No expiration para que el token no expire y se mantenga siempre disponible para nuestro proyecto.



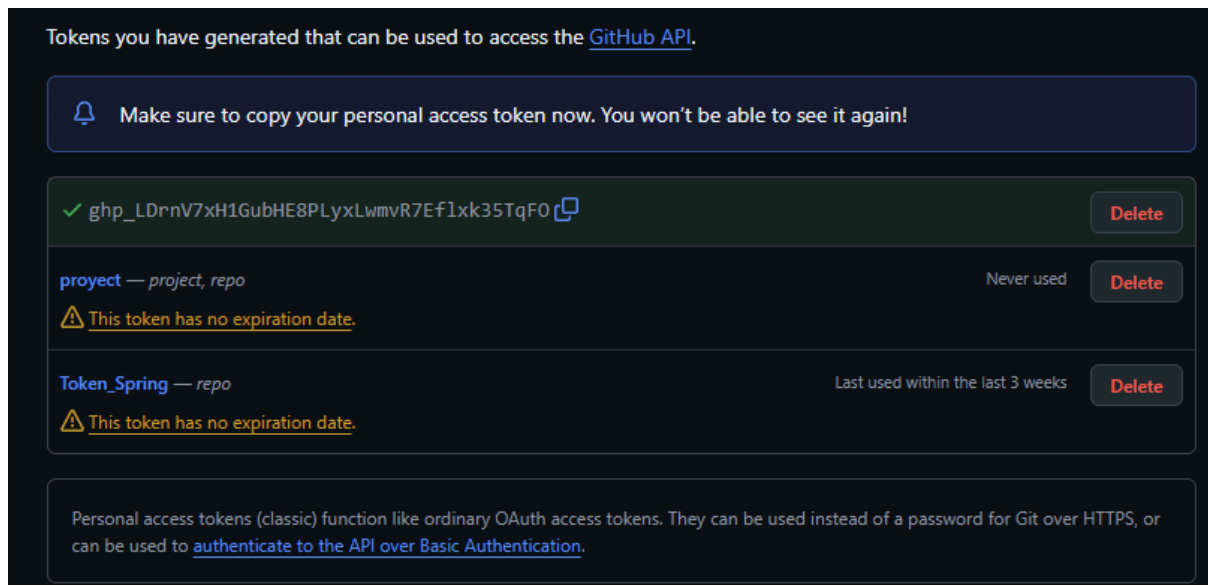
Seleccionamos la casilla repo.



Luego bajamos hasta encontrar el botón Generate token y lo seleccionamos.



Ya estará generado el token que utilizaremos para nuestra contraseña dentro de nuestro repositorio.



8.3 Git Add y Git Commit

Estos comandos nos servirán para agregar o registrar los cambios realizados a los archivos del repositorio.

Con el comando `git add .` nosotros agregaremos los archivos a una “zona de preparación” o “cola de preparación” para indicar que estos archivos pronto serán guardados y registrados en el historial de versiones del repositorio.

```
nicho@kingniki MINGW64 /g/2025/Full Stack I/Ev2_EduTech (test)
● $ git add .
```

Con `git commit -m "mensaje descriptivo"` registramos de forma definitiva los archivos que están en la “cola de preparación” y se guardarán en el historial de versiones para github

```
nicho@kingniki MINGW64 /g/2025/Full Stack I/Ev2_EduTech (test)
● $ git commit -m "implementacion de entidad Alumno"
[test 655bc29] implementacion de entidad Alumno
6 files changed, 261 insertions(+), 1 deletion(-)
create mode 100644 proyect/src/main/java/com/edutech_innovators/proyect/controllers/AlumnoController.java
create mode 100644 proyect/src/main/java/com/edutech_innovators/proyect/entities/Alumno.java
create mode 100644 proyect/src/main/java/com/edutech_innovators/proyect/repository/AlumnoRepository.java
create mode 100644 proyect/src/main/java/com/edutech_innovators/proyect/services/AlumnoService.java
create mode 100644 proyect/src/main/java/com/edutech_innovators/proyect/services/AlumnoServiceImpl.java
```

En este caso en particular agregamos un cambio en el cual implementamos la entidad Alumno y configuramos el servicio ApiREST para Alumno. Se guardaron los cambios para próximamente llevarlos a un repositorio remoto en GitHub con el comando `git push`.

8.4 Git Remote, Git Branch, Git Checkout y Git Push

Estos comandos nos servirán para acceder y agregar el repositorio del proyecto directamente a la web de GitHub en un repositorio remoto. Podremos agregar, acceder y visualizar las diferentes ramas que usaremos para el desarrollo de nuestro proyecto

Empezando por el comando `git remote add origin url_del_repositorio` nos permitirá conectar nuestro repositorio local hacia un repositorio remoto ubicado en GitHub

```
nicho@kingniki MINGW64 /g/2025/Full Stack I/Ev2_EduTech (test)
$ git remote add origin https://github.com/NicolasCarrasco7261/Evaluacion_Parcial_2_EduTech.git
```

Luego utilizamos el comando `git branch -M main` para cambiar el nombre de la rama por defecto y dejarlo con el nombre “main” ya que las herramientas automáticas de GitHub asumen que la rama principal se llama “main” y es importante recordar que con este mismo comando (sin -M) podemos crear ramas

```
nicho@kingniki MINGW64 ~/Desktop/Ev2_EduTech (master)
$ git branch -M main
```

Por último para guardar los cambios (commits) en la rama principal y subirlos al repositorio remoto en GitHub por primera vez, utilizaremos el comando `git push -u origin main`. Luego cuando nosotros hagamos cualquier cambio, podremos seguir utilizando este mismo comando para subirlo al repositorio remoto, pero no será necesario colocarle el -u

```
nicho@kingniki MINGW64 ~/Desktop/Ev2_EduTech (main)
$ git push -u origin main
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 209 bytes | 209.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/NicolasCarrasco7261/Examen_Transversal.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

Si nosotros queremos cambiar la rama que estamos utilizando actualmente por otra, desde la terminal bash utilizaremos el comando `git checkout nombre_rama`

```
nicho@kingniki MINGW64 /g/2025/Full Stack I/Ev2_EduTech (test)
$ git checkout test2
Switched to branch 'test2'
Your branch is up to date with 'origin/test2'.

nicho@kingniki MINGW64 /g/2025/Full Stack I/Ev2_EduTech (test2)
$
```

Si queremos crear otra rama y a su vez cambiarnos a esa, solo debemos usar el comando `git checkout -b nombre_rama` y solo se verá creada la rama en la página de GitHub cuando nosotros hagamos un `git push` previamente.

```
nicho@kingniki MINGW64 /g/2025/Full Stack I/Ev2_EduTech (test)
$ git checkout -b test4
Switched to a new branch 'test4'

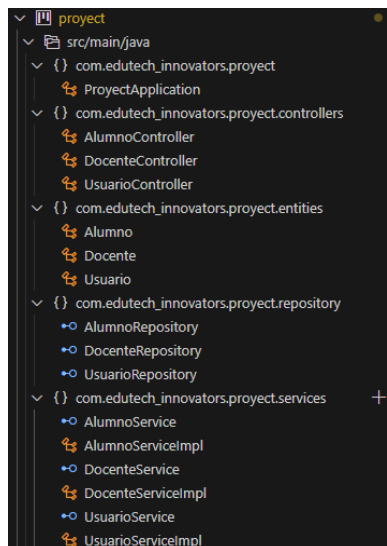
nicho@kingniki MINGW64 /g/2025/Full Stack I/Ev2_EduTech (test4)
$
```

8.5 Git Merge

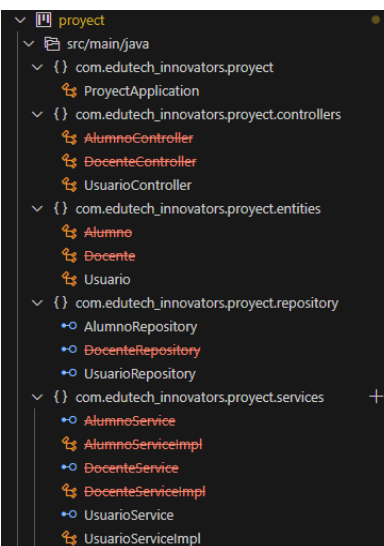
Este comando de git es muy importante en un proyecto colaborativo, ya que con este comando podemos traspasar archivos desde una rama hacia otra utilizando git merge.

Si nosotros colocamos en la terminal bash el comando `git merge rama_origen rama_destino`, se traspasarán los archivos que están en la rama_origen hacia la rama_destino siempre y cuando estemos trabajando dentro de la rama_destino. Esta rama debe ser la que está más desactualizada y al hacer el merge se fusionarán los archivos.

Rama test:



Rama test2:



Ejecutamos el comando `git merge test test2` para hacer la fusión desde la rama test hacia la rama test2 y se agregan los archivos.

```

nicho@kingn1k1 MINGW64 /g/2025/Full Stack 1/Ev2_EduTech (test2)
$ git merge test2
Merge made by the 'ort' strategy.
 .vscode/launch.json          | 14 +++
 .vscode/settings.json        | 3 ++
 project/pom.xml              | 17 +++
 .../project/controllers/AlumnoController.java | 88 ++++++
 .../project/controllers/DocenteController.java | 89 ++++++
 .../project/controllers/UsuarioController.java | 84 ++++++
 .../project/entities/Alumno.java | 84 ++++++
 .../project/entities/Docente.java | 80 ++++++
 .../project/entities/Usuario.java | 28 +++++
 .../project/repository/AlumnoRepository.java | 9 +++
 .../project/repository/DocenteRepository.java | 9 +++
 .../project/repository/UsuarioRepository.java | 18 +++
 .../project/services/AlumnoService.java | 19 +++++
 .../project/services/AlumnoServiceImpl.java | 68 ++++++
 .../project/services/DocenteService.java | 22 +++++
 .../project/services/DocenteServiceImpl.java | 59 ++++++
 .../project/services/UsuarioService.java | 20 +++++
 .../project/services/UsuarioServiceImpl.java | 57 ++++++
 18 files changed, 736 insertions(+), 16 deletions(-)
 create mode 100644 .vscode/launch.json
 create mode 100644 project/src/main/java/com/edutech_innovators/project/controllers/AlumnoController.java
 create mode 100644 project/src/main/java/com/edutech_innovators/project/controllers/DocenteController.java
 create mode 100644 project/src/main/java/com/edutech_innovators/project/controllers/UsuarioController.java
 create mode 100644 project/src/main/java/com/edutech_innovators/project/entities/Alumno.java
 create mode 100644 project/src/main/java/com/edutech_innovators/project/entities/Docente.java
 create mode 100644 project/src/main/java/com/edutech_innovators/project/repository/AlumnoRepository.java
 create mode 100644 project/src/main/java/com/edutech_innovators/project/repository/DocenteRepository.java
 create mode 100644 project/src/main/java/com/edutech_innovators/project/repository/UsuarioRepository.java
 create mode 100644 project/src/main/java/com/edutech_innovators/project/services/AlumnoService.java
 create mode 100644 project/src/main/java/com/edutech_innovators/project/services/AlumnoServiceImpl.java
 create mode 100644 project/src/main/java/com/edutech_innovators/project/services/DocenteService.java
 create mode 100644 project/src/main/java/com/edutech_innovators/project/services/DocenteServiceImpl.java
 create mode 100644 project/src/main/java/com/edutech_innovators/project/services/UsuarioService.java
 create mode 100644 project/src/main/java/com/edutech_innovators/project/services/UsuarioServiceImpl.java

```

Si ingresamos al final del comando un comentario y realizamos el push hacia la rama, Git ingresa el mensaje del merge como si fuera un commit, pero en este caso generará automáticamente uno

test2
 3 Branches
 0 Tags

This branch is **7 commits ahead** of **main**.

NicolasCarrasco7261 Merge branch 'test' into test2
 36af9d8 · 15 minutes ago
🕒 9 Commits

.vscode

Implementacion de entidad Usuario

2 days ago

project

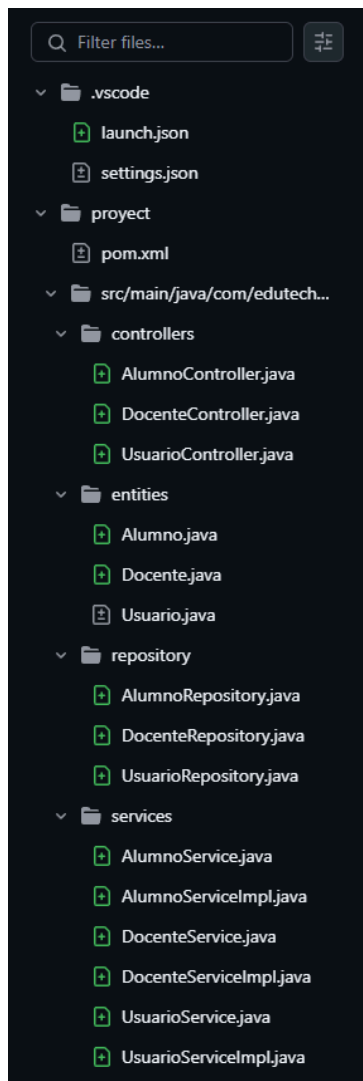
Merge branch 'test' into test2

15 minutes ago

Commit 36af9d8

NicolasCarrasco7261 committed 18 minutes ago

Merge branch 'test' into test2
 test2



9. Conclusión

Este informe ha evidenciado el avance que hemos realizado en nuestro proyecto para la empresa EduTech Innovators. Iniciamos este informe con la estructura del proyecto, donde ahondamos en las partes que componen nuestro sistema como los packages, los recursos, las dependencias en pom, etc. Luego pasamos al apartado de la base de datos, donde explicamos las herramientas que usamos, para qué sirven y cómo funcionan.

Más adelante, revisamos la implementación de los servicios donde utilizamos el software Postman para las solicitudes o requerimientos al sistema trayendo, creando, modificando y eliminando datos. Vimos además la ruta que siguen estas solicitudes desde Postman hasta que la información se almacena en la base de datos, pasando por la API Rest Controller, los servicios y el repositorio.

Luego hicimos un recorrido por los comandos Git que nos permiten la administración de los archivos y directorios del proyecto subiéndolos finalmente a la plataforma GitHub, donde quedarán disponibles para continuar con su administración más adelante.